

Московский Авиационный Институт  
(Национальный Исследовательский Университет)  
Институт №8 “Компьютерные науки и прикладная математика”  
Кафедра №806 “Вычислительная математика и программирование”

**Лабораторная работа №4 по курсу**  
**«Операционные системы»**

Группа: М8О-213Б-23

Студент: Пономарев А.А

Преподаватель: Бахарев В.Д.

Оценка: \_\_\_\_\_

Дата: 21.12.24

Москва, 2024

# Постановка задачи

## Вариант 3.

Исследовать два аллокатора памяти: необходимо реализовать два алгоритма аллокации памяти и сравнить их по следующим характеристикам: –

- Фактор использования
- Скорость выделения блоков
- Скорость освобождения блоков
- Простота использования аллокатора

Требуется создать две динамические библиотеки, реализующие два аллокатора, соответственно. Библиотеки загружаются в память с помощью интерфейса ОС (dlopen / LoadLibrary) для работы с динамическими библиотеками. Выбор библиотеки, реализующей аллокатор, осуществляется чтением первого аргумента при запуске программы (argv[1]). Этот аргумент должен содержать путь до динамической библиотеки (относительный или абсолютный). Аллокаторы – метод двойников и алгоритм списки свободных блоков(первое подходящее).

## Общий метод и алгоритм решения

Использованные системные вызовы:

- int write(int fd, void\* buf, size\_t count); – записывает count байт из buf в fd.
- void \*mmap(void addr, size\_t length, int prot, int flags, int fd, off\_t offset); – выполняет отображение файла или устройства на память.
- int munmap(void addr, size\_t length); – удаляет отображение файла или устройства на память.

Общий метод и алгоритм решения, который использован в вашем проекте, состоит из создания динамического распределителя памяти, который поддерживает два типа аллокаторов — **Buddy Allocator** и **FreeList Allocator**. Ваше приложение загружает эти аллокаторы через динамические библиотеки и позволяет выбрать, какой из них использовать.

### 1. Алгоритм решения:

### 2. Динамическое подключение библиотек (main.c):

- В начале программы проверяется, был ли передан путь к динамической библиотеке аллокатора. Если путь не передан, используется стандартный аллокатор.
- Если путь передан, используется функция `dlopen` для загрузки библиотеки и `dlsym` для извлечения указателей на функции аллокатора (создание, выделение памяти, освобождение и уничтожение).
- Если загрузка библиотеки не удалась, используется стандартный аллокатор.

### 3. Создание аллокатора (allocator\_create):

- При создании аллокатора (как в `buddy.c`, так и в `free_list.c`) выделяется память для структуры аллокатора и инициализируются свободные блоки.
- Для **Buddy Allocator** создается один большой блок, который может быть разделен на более мелкие блоки с помощью алгоритма "buddy".
- Для **FreeList Allocator** создается связанный список, в котором каждый элемент представляет собой свободный блок памяти.

### 4. Аллокация памяти (allocator\_alloc):

- В функции аллокации происходит поиск подходящего блока памяти.
- В **Buddy Allocator** блоки делятся на два, если они слишком большие, пока не достигнут нужного размера. Этот процесс повторяется, пока не будет найден подходящий блок.
- В **FreeList Allocator** происходит поиск подходящего блока в списке свободных блоков. Если блок слишком большой, он делится на два — один блок возвращается пользователю, а второй остаётся в списке.
- В обоих случаях, если не удастся найти подходящий блок, возвращается `NULL`.

### 5. Освобождение памяти (allocator\_free):

- В **Buddy Allocator** освобожденный блок пытается "сливаться" с его "buddy" (соседним блоком). Если соседний блок свободен, два блока сливаются в один. Этот процесс может повторяться рекурсивно, если происходит слияние нескольких блоков.
- В **FreeList Allocator** освобожденный блок добавляется обратно в список свободных блоков.

### 6. Завершение работы (allocator\_destroy):

- После завершения работы с аллокатором вызывается функция `allocator_destroy`, которая очищает внутренние структуры аллокатора.

## 7. Описание алгоритмов для каждого типа аллокатора:

### Buddy Allocator (`buddy.c`):

- Используется принцип "buddy" (соседей), когда память делится на блоки, и каждый блок имеет своего "соседа" (buddy), с которым он может сливаться, если оба блока свободны.
- Размер блока всегда является степенью двойки, что упрощает управление памятью.
- Алгоритм разбивает блоки пополам, пока не находит подходящий размер для запроса.

### FreeList Allocator (`free_list.c`):

- Используется связанный список свободных блоков, где каждый элемент содержит информацию о размере блока и указатель на следующий блок.
- Когда блок освобождается, он добавляется в начало списка. Когда происходит запрос на выделение памяти, система ищет первый подходящий блок и возвращает его.
- При необходимости свободный блок может быть разделен на два меньших блока.

## 8. Пошаговый алгоритм работы программы:

1. Программа загружает динамическую библиотеку с аллокатором или использует стандартный аллокатор.
2. Выделяется область памяти с помощью `mmap`, и создается экземпляр аллокатора.
3. Выполняется несколько операций выделения памяти (`allocator_alloc`).
4. После использования памяти, она освобождается с помощью `allocator_free`.
5. Программа завершает работу, вызывая `allocator_destroy`, чтобы освободить ресурсы.

Ваш код позволяет гибко переключаться между различными методами аллокации памяти, загружая соответствующие библиотеки, что предоставляет удобство выбора аллокатора в зависимости от требований к производительности или особенностей задачи

## Код программы

### main.c

```
#include <dlfcn.h>
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/mman.h>
#include <unistd.h>

typedef struct Allocator {
```

```

    void (*allocator_create)(void *addr, size_t size);
    void (*allocator_alloc)(void *allocator, size_t size);
    void (*allocator_free)(void *allocator, void *ptr);
    void (*allocator_destroy)(void *allocator);
} Allocator;

void *standard_allocator_create(void *memory, size_t size) {
    (void)size;
    (void)memory;
    return memory;
}

void *standard_allocator_alloc(void *allocator, size_t size) {
    uint32_t *memory = mmap(NULL, size + sizeof(uint32_t), PROT_READ
| PROT_WRITE,
                            MAP_SHARED | MAP_ANONYMOUS, -1, 0);
    if (memory == MAP_FAILED) {
        return NULL;
    }
    *memory = (uint32_t)(size + sizeof(uint32_t));
    return memory + 1;
}

void standard_allocator_free(void *allocator, void *memory) {
    if (memory == NULL)
        return;
    uint32_t *mem = (uint32_t *)memory - 1;
    munmap(mem, *mem);
}

void standard_allocator_destroy(void *allocator) {
    (void)allocator;
}

void load_allocator(const char *library_path, Allocator *allocator) {
    void *library = dlopen(library_path, RTLD_LOCAL | RTLD_NOW);
    if (!library) {
        // Если не удалось загрузить библиотеку, используем
стандартные функции
        char message[] = "WARNING: failed to load shared library\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        allocator->allocator_create = standard_allocator_create;
        allocator->allocator_alloc = standard_allocator_alloc;
        allocator->allocator_free = standard_allocator_free;
        allocator->allocator_destroy = standard_allocator_destroy;
        return;
    }

    // Загружаем функции из библиотеки
    allocator->allocator_create = dlsym(library, "allocator_create");
    allocator->allocator_alloc = dlsym(library, "allocator_alloc");
    allocator->allocator_free = dlsym(library, "allocator_free");
    allocator->allocator_destroy = dlsym(library,
"allocator_destroy");

```

```

        if (!allocator->allocator_create || !allocator->allocator_alloc
|| !allocator->allocator_free || !allocator->allocator_destroy) {
            const char msg[] = "Error: failed to load all allocator
functions\n";
            write(STDERR_FILENO, msg, sizeof(msg) - 1);
            dlclose(library);
        }
    }

int main(int argc, char **argv) {
    const char *library_path = (argc > 1) ? argv[1] : NULL;

    // Статически выделяем память для структуры Allocator
    Allocator allocator;
    load_allocator(library_path, &allocator);

    size_t size = 4104;
    void *addr = mmap(NULL, size, PROT_READ | PROT_WRITE, MAP_PRIVATE
| MAP_ANONYMOUS, -1, 0);
    if (addr == MAP_FAILED) {
        char message[] = "mmap failed\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        return EXIT_FAILURE;
    }

    void *allocator_instance = allocator.allocator_create(addr,
size);
    if (!allocator_instance) {
        char message[] = "Failed to initialize allocator\n";
        write(STDERR_FILENO, message, sizeof(message) - 1);
        munmap(addr, size);
        return EXIT_FAILURE;
    }

    char start_message[] = "Allocator initialized\n";
    write(STDOUT_FILENO, start_message, sizeof(start_message) - 1);

    void *block1 = allocator.allocator_alloc(allocator_instance,
256);
    void *block2 = allocator.allocator_alloc(allocator_instance,
1023);

    if (block1 == NULL || block2 == NULL) {
        char alloc_fail_message[] = "Memory allocation failed\n";
        write(STDERR_FILENO, alloc_fail_message,
sizeof(alloc_fail_message) - 1);
    } else {
        char alloc_success_message[] = "Memory allocated
successfully\n";
        write(STDOUT_FILENO, alloc_success_message,
sizeof(alloc_success_message) - 1);
    }

    char buffer[64];
    snprintf(buffer, sizeof(buffer), "Block 1 address: %p\n",
block1);

```

```

    write(STDOUT_FILENO, buffer, strlen(buffer));
    snprintf(buffer, sizeof(buffer), "Block 2 address: %p\n",
block2);
    write(STDOUT_FILENO, buffer, strlen(buffer));

    allocator.allocator_free(allocator_instance, block1);
    allocator.allocator_free(allocator_instance, block2);

    char free_message[] = "Memory freed\n";
    write(STDOUT_FILENO, free_message, sizeof(free_message) - 1);

    allocator.allocator_destroy(allocator_instance);
    munmap(addr, size);

    char exit_message[] = "Program exited successfully\n";
    write(STDOUT_FILENO, exit_message, sizeof(exit_message) - 1);

    return EXIT_SUCCESS;
}

```

Buddy.c

```

#include <stddef.h>
#include <stdint.h> // Для использования uintptr_t

// Структура блока
typedef struct Block {
    size_t size;           // Размер блока
    struct Block *next;    // Указатель на следующий свободный блок
    int free;              // Флаг, указывающий, свободен ли блок
} Block;

// Функция для нахождения соседнего блока с использованием XOR
Block *find_buddy(Block *block_to_free) {
    // Преобразуем указатель на блок в целое число
    uintptr_t block_addr = (uintptr_t)block_to_free;

    // Вычисляем адрес соседнего блока с помощью операции XOR
    uintptr_t buddy_addr = block_addr ^ block_to_free->size;

    // Преобразуем адрес соседнего блока обратно в указатель
    Block *buddy = (Block *)buddy_addr;

    return buddy;
}

typedef struct Allocator {
    Block *free_blocks;
} Allocator;

void *allocator_alloc(Allocator *allocator, size_t size) {
    Block *current = allocator->free_blocks;
    Block *prev = NULL;
    size_t required_size = 1;
    while (required_size < size) {
        required_size <= 1;
    }
}

```

```

size = required_size;

while (current) {
    if (current->free && current->size >= size) {
        while (current->size > size) {
            // Разбиваем блок на два меньших
            // Находим адрес для нового блока
            Block *buddy = (Block *)((uintptr_t)current +
current->size / 2);
            buddy->size = current->size / 2;
            buddy->free = 1;

            current->size /= 2;

            // Связываем buddy в список
            buddy->next = current->next;
            current->next = buddy;
        }
        current->free = 0;

        // Убираем блок из списка свободных
        if (prev) {
            prev->next = current->next;
        } else {
            allocator->free_blocks = current->next;
        }

        // Возвращаем указатель на данные после заголовка
        return (char *)current + sizeof(Block);
    }

    prev = current;
    current = current->next;
}
return NULL;
}

void allocator_free(Allocator *allocator, void *ptr) {
    Block *block_to_free = (Block *)((char *)ptr - sizeof(Block));
    block_to_free->free = 1;

    Block *buddy = find_buddy(block_to_free);

    // Проверяем, свободен ли соседний блок
    if (buddy && buddy->free && buddy->size == block_to_free->size) {
        // Сливаем два блока
        if (buddy > block_to_free) {
            // Сосед правый, обновляем текущий блок
            block_to_free->size *= 2;
            block_to_free->next = buddy->next;
        } else {
            // Сосед левый, обновляем buddy
            buddy->size *= 2;
            buddy->next = block_to_free->next;
        }
    }
    // Рекурсивно пытаемся слить с соседями

```



```

        allocator_free(allocator, (char *)buddy + sizeof(Block));
    } else {
        // Если не можем слить, добавляем блок в список свободных
        block_to_free->next = allocator->free_blocks;
        allocator->free_blocks = block_to_free;
    }
}

Allocator *allocator_create(void *memory, size_t size) {
    Allocator *allocator = (Allocator *)memory;
    allocator->free_blocks = (Block *) (memory + sizeof(Allocator));
    allocator->free_blocks->size = size - sizeof(Allocator);
    allocator->free_blocks->free = 1;
    allocator->free_blocks->next = NULL;

    return allocator;
}

void allocator_destroy(Allocator *allocator) {
    allocator->free_blocks = NULL;
}

Freelist.c
#include <stddef.h>
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>

typedef struct FreeListNode {
    struct FreeListNode *next; // Указатель на следующий блок
    size_t size;               // Размер блока памяти
} FreeListNode;

typedef struct Allocator {
    void *memory;
    size_t total_size;
    FreeListNode *free_list;
} Allocator;

Allocator* allocator_create(void *const memory, const size_t size) {
    if (memory == NULL || size == 0) {
        return NULL;
    }

    Allocator *allocator = (Allocator *) memory;
    allocator->total_size = size - sizeof(Allocator);
    allocator->free_list = (FreeListNode *) ((uintptr_t)memory +
sizeof(Allocator));

    allocator->free_list->next = NULL;
    allocator->free_list->size = allocator->total_size;

    return allocator;
}

void allocator_destroy(Allocator *const allocator) {

```

```

    if (allocator != NULL) {
        allocator->memory = NULL;
        allocator->free_list = NULL;
    }
}

void* allocator_alloc(Allocator *const allocator, const size_t size)
{
    if (allocator == NULL || size == 0) {
        return NULL;
    }
    FreeListNode **prev = &allocator->free_list;
    FreeListNode *curr = *prev;

    while (curr != NULL) {
        if (curr->size >= size) {
            if (curr->size > size + sizeof(FreeListNode)) {
                // Создаем новый блок, оставшийся после выделения
                FreeListNode *new_block = (FreeListNode
*)((uintptr_t)curr + size);
                new_block->size = curr->size - size;
                new_block->next = curr->next;

                curr->size = size;
                curr->next = new_block;
            }

            // Убираем блок из списка свободных блоков
            *prev = curr->next;

            // Возвращаем указатель на выделенный блок (после
свободного списка)
            return (void *)((uintptr_t)curr + sizeof(FreeListNode));
        }
        prev = &curr->next;
        curr = curr->next;
    }
    return NULL;
}

void allocator_free(Allocator *const allocator, void *const ptr) {
    if (allocator == NULL || ptr == NULL) {
        return;
    }
    FreeListNode *block_to_free = (FreeListNode *)((uintptr_t)ptr -
sizeof(FreeListNode));

    // Добавляем освобожденный блок в начало списка
    block_to_free->next = allocator->free_list;
    allocator->free_list = block_to_free;
}

```

## Протокол работы программы

### Тестирование:

```
artem@artem-VirtualBox:~/MAI_OS/lab04/src$ ./main ./buddyallocator.so
```

```
Allocator initialized
```

```
Memory allocated successfully
```

```
Block 1 address: 0x7f7e39022020
```

```
Block 2 address: 0x7f7e39022420
```

```
Memory freed
```

```
Program exited successfully
```

```
artem@artem-VirtualBox:~/MAI_OS/lab04/src$ ./main ./buddyalloca.so
```

```
WARNING: failed to load shared library
```

```
Allocator initialized
```

```
Memory allocated successfully
```

```
Block 1 address: 0x7f7e9e7d6004
```

```
Block 2 address: 0x7f7e9e7a7004
```

```
Memory freed
```

```
Program exited successfully
```

```
artem@artem-VirtualBox:~/MAI_OS/lab04/src$ ./main ./buddyallocator.so
```

```
Allocator initialized
```

```
Memory allocated successfully
```

```
Block 1 address: 0x7fb5271020
```

```
Block 2 address: 0x7fb5271420
```

```
Memory freed
```

```
Program exited successfully
```

### **Strace:**

```
strace ./main ./buddyallocator.so
```

```
execve("./main", ["/main", "/buddyallocator.so"], 0x7ffe53879208 /* 49 vars */) = 0
```

```
brk(NULL) = 0x561e39db2000
```

```
arch_prctl(0x3001 /* ARCH_??? */, 0x7ffcb1395da0) = -1 EINVAL (Недопустимый аргумент)
```

```
access("/etc/ld.so.preload", R_OK) = -1 ENOENT (Нет такого файла или каталога)
```

```
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
```

```
fstat(3, {st_mode=S_IFREG|0644, st_size=70654, ...}) = 0
```

```
mmap(NULL, 70654, PROT_READ, MAP_PRIVATE, 3, 0) = 0x7f142a42a000
```

```
close(3) = 0
```

```

openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libdl.so.2", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\22\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=18848, ...}) = 0
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7f142a428000
mmap(NULL, 20752, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f142a422000
mmap(0x7f142a423000, 8192, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1000) = 0x7f142a423000
mmap(0x7f142a425000, 4096, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000) = 0x7f142a425000
mmap(0x7f142a426000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3000) = 0x7f142a426000
close(3) = 0
openat(AT_FDCWD, "/lib/x86_64-linux-gnu/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\3\0\0\0\0\0\0\3\0>\0\1\0\0\0\300A\2\0\0\0\0"..., 832) = 832
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
pread64(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0", 32, 848) = 32
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\7\2C\n\357_\243\335\2449\206V>\237\374\304"...,
68, 880) = 68
fstat(3, {st_mode=S_IFREG|0755, st_size=2029592, ...}) = 0
pread64(3, "\6\0\0\0\4\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0@\0\0\0\0\0\0"..., 784, 64) = 784
pread64(3, "\4\0\0\0\20\0\0\0\5\0\0\0GNU\0\2\0\0\300\4\0\0\0\3\0\0\0\0\0\0", 32, 848) = 32
pread64(3, "\4\0\0\0\24\0\0\0\3\0\0\0GNU\0\7\2C\n\357_\243\335\2449\206V>\237\374\304"...,
68, 880) = 68
mmap(NULL, 2037344, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f142a230000
mmap(0x7f142a252000, 1540096, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x22000) = 0x7f142a252000
mmap(0x7f142a3ca000, 319488, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x19a000) = 0x7f142a3ca000
mmap(0x7f142a418000, 24576, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1e7000) = 0x7f142a418000
mmap(0x7f142a41e000, 13920, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0x7f142a41e000
close(3) = 0
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1,
0) = 0x7f142a22d000

```

```

arch_prctl(ARCH_SET_FS, 0x7f142a22d740) = 0
mprotect(0x7f142a418000, 16384, PROT_READ) = 0
mprotect(0x7f142a426000, 4096, PROT_READ) = 0
mprotect(0x561e115cb000, 4096, PROT_READ) = 0
mprotect(0x7f142a469000, 4096, PROT_READ) = 0
munmap(0x7f142a42a000, 70654) = 0
brk(NULL) = 0x561e39db2000
brk(0x561e39dd3000) = 0x561e39dd3000
openat(AT_FDCWD, "./buddyallocator.so", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\3\0>\0\1\0\0\0\20\0\0\0\0\0"... , 832) = 832
fstat(3, {st_mode=S_IFREG|0775, st_size=18520, ...}) = 0
getcwd("/home/artem/MAI_OS/lab04/src", 128) = 29
mmap(NULL, 16432, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) =
0x7f142a437000
mmap(0x7f142a438000, 4096, PROT_READ|PROT_EXEC,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1000) = 0x7f142a438000
mmap(0x7f142a439000, 4096, PROT_READ,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7f142a439000
mmap(0x7f142a43a000, 8192, PROT_READ|PROT_WRITE,
MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x2000) = 0x7f142a43a000
close(3) = 0
mprotect(0x7f142a43a000, 4096, PROT_READ) = 0
mmap(NULL, 4104, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS,
-1, 0) = 0x7f142a435000
write(1, "Allocator initialized\n", 22Allocator initialized
) = 22
write(1, "Memory allocated successfully\n", 30Memory allocated successfully
) = 30
write(1, "Block 1 address: 0x7f142a435020\n", 32Block 1 address: 0x7f142a435020
) = 32
write(1, "Block 2 address: 0x7f142a435420\n", 32Block 2 address: 0x7f142a435420
) = 32
write(1, "Memory freed\n", 13Memory freed
) = 13

```

```
munmap(0x7f142a435000, 4104) = 0
```

```
write(1, "Program exited successfully\n", 28Program exited successfully  
) = 28
```

```
exit_group(0) = ?
```

```
+++ exited with 0 +++
```

## Вывод

В ходе выполнения этой лабораторной работы я бы научился работать с динамическими библиотеками в С, используя `dlopen` и `dlsym` для загрузки и использования внешних функций во время выполнения. Также я бы освоил реализацию двух типов аллокаторов памяти — **Buddy Allocator** и **FreeList Allocator**, научившись эффективно управлять памятью с использованием различных алгоритмов выделения и освобождения памяти, таких как слияние блоков и управление свободными блоками. Это помогло бы мне понять основы системного программирования и оптимизации работы с памятью